

Evaluation Pipeline — Son of Anton

Overview

The evaluation pipeline measures Son Of Anton's performance across three dimensions:

1. **Chat Groundedness** — Does Son Of Anton answer questions accurately without hallucination?
2. **Voice Quality** — What is Son Of Anton's latency and call handling quality?
3. **Failure Modes** — What patterns of errors occur and how can they be fixed?

Files

- **golden_qa.json** — 50-question golden dataset (resume, github, scheduling, context, adversarial)
- **run_evals.py** — Main evaluation runner
- **analyze_failures.py** — Failure mode detector and analyzer
- **results.json** — Detailed results from latest eval run
- **eval_report_summary.json** — High-level summary metrics for PDF report
- **failure_modes.json** — Detailed failure mode analysis

Running the Evaluation Pipeline

Step 1: Ensure Backend is Running

```
cd /home/maverick/SonOfAnton
source backend/.venv/bin/activate
uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

Step 2: Run the Chat Q&A Evals

PROF

```
python -m backend.evals.run_evals
```

This will:

1. Load all 50 questions from golden_qa.json
2. Run each question through the Anton orchestrator graph
3. Judge each answer with GPT-4o-mini (grounded/honest/on_character)
4. Compute per-domain metrics (resume, github, scheduling, context, adversarial)
5. Generate **results.json** with detailed Q&A results
6. Analyze failure modes and integrate into **eval_report_summary.json**

Step 3: Review Output Files

results.json — Per-question details

```
[
  {
    "id": "qa_001",
    "question": "What is Rehan's email address?",
    "expected": ["mdrehan05.2006@gmail.com"],
    "answer": "...",
    "latency_ms": 1250,
    "verdict": {
      "grounded": true,
      "honest": true,
      "on_character": true,
      "notes": "..."
    },
    "domain": "resume"
  },
  ...
]
```

eval_report_summary.json — Summary report for PDF

```
{
  "chat": {
    "total_questions": 50,
    "hallucination_rate": 0.0,
    "honesty_rate": 0.95,
    "on_character_rate": 1.0,
    "avg_latency_ms": 1200,
    "by_domain": {
      "resume": {
        "total": 15,
        "grounded": 15,
        "avg_latency_ms": 1100
      },
      ...
    },
    "retrieval": {
      "precision": 0.85,
      "recall": { ... }
    }
  },
  "voice": {
    "first_response_latency_ms": 800,
    "booking_success_rate": 0.9,
    "test_calls_n": 10
  },
  "failure_modes": [ ... ],
  "tradeoff": { ... },

```

```
"future_2_weeks": "..."  
}
```

Metrics Explained

Chat Groundedness

- **hallucination_rate** = (questions with grounded=false) / total
 - Target: < 5%
 - Measure of factual accuracy
- **honesty_rate** = (answers that admit uncertainty) / total
 - Target: > 90% on questions where info is missing
 - Measure of truthfulness
- **on_character_rate** = (responses staying in Anton role) / total
 - Target: 100%
 - Measure of prompt-injection resistance
- **avg_latency_ms** = average time to generate answer
 - Target: < 2000ms
 - Per-domain targets: resume=1000ms, github=1200ms, scheduling=800ms

Voice Quality

- **first_response_latency_ms** = time from call-start to first audio token
 - Target: < 1000ms
 - Critical for user experience
- **booking_success_rate** = (test calls that complete booking) / total_test_calls
 - Target: > 95%
 - Measure of scheduling flow robustness

—
PROF

Failure Modes

The analyzer detects three primary failure mode categories:

1. Router Misclassification

- Symptom: Scheduling queries routed to knowledge, or vice versa
- Root cause: Keyword collision (e.g., "3 works for me" triggers knowledge classifier)
- Metric: Count of queries with wrong intent classification

2. Retrieval Threshold Edge Cases

- Symptom: Answered grounded but from <2 chunks, or not grounded despite chunk retrieval
- Root cause: RETRIEVAL_SCORE_THRESHOLD (0.35) is too aggressive or lenient
- Metric: Count of threshold-related retrieval failures

3. Latency Spikes

- Symptom: Certain query patterns cause > 2x average latency
- Root cause: LLM fallback in router, expensive reranking
- Metric: Count of queries with latency > 2x average

Interpretation Guide

< 5% Hallucination Rate

- ✓ **Good** — Anton is factually reliable
- × **Bad** — Manual review of failed cases needed

On-Character Rate = 100%

- ✓ **Good** — Prompt injections are handled
- × **Bad** — Jailbreak examples, review manually

Routing Accuracy > 95%

- ✓ **Good** — Router is stable
- × **Bad** — Check keyword lists in router_node.py

First Response Latency < 1000ms

- ✓ **Good** — User experience is smooth
- × **Bad** — Profile LLM calls and embedding cache hits

Common Issues & Fixes

High Hallucination Rate (> 10%)

1. Check retrieval scores — is threshold too high?
2. Review failed questions in results.json
3. Check if GitHub fallback is stale (run ingestion)
4. Increase trust weights for authoritative sources

Low Honesty Rate (< 80%)

1. Review "honest" verdict failures in results.json
2. Anton should say "I don't have enough information" more often
3. Check response_node.py for uncertainty expression

High Latency (avg > 2000ms)

1. Profile the router — is LLM fallback happening too often?

2. Check embedding generation time (get_embedder)
3. Check if Redis cache is being hit (cache.py logs)

Advanced: Customizing Thresholds

Adjust RETRIEVAL_SCORE_THRESHOLD

Edit `backend/config.py`:

```
RETRIEVAL_SCORE_THRESHOLD: float = 0.35 # ← change here
```

Lower (e.g., 0.25) → More aggressive retrieval, higher hallucination risk

Higher (e.g., 0.45) → Stricter retrieval, higher "I don't know" rate

Adjust Domain Trust Weights

Edit `backend/knowledge/retriever.py`:

```
TRUST_WEIGHTS = {  
    "personal": {"resume": 1.3, "linkedin": 1.1, "github": 0.8},  
    "projects": {"resume": 1.0, "linkedin": 0.8, "github": 1.4},  
    ...  
}
```

Higher weight → Source is prioritized

Lower weight → Source is deprioritized

Logging & Database

All eval runs are logged to PostgreSQL:

- `eval_logs` — Chat Q&A verdicts, latencies, hallucination flags
- `voice_eval_logs` — Call quality metrics (first_response_latency, transcript, recording)

Query examples:

```
-- Hallucination rate by domain  
SELECT source_domain,  
       COUNT(*) as total,  
       SUM(CASE WHEN is_hallucination THEN 1 ELSE 0 END) as  
hallucinations  
FROM eval_logs  
GROUP BY source_domain;  
  
-- Average latency by source  
SELECT source_domain, AVG(latency_ms) as avg_latency
```

```
FROM eval_logs  
GROUP BY source_domain;
```

Future Improvements (2 weeks)

- ☐ A/B test retrieval score thresholds per domain
- ☐ Implement adaptive router confidence (currently hardcoded 0.7)
- ☐ Add phone number capture in scheduling flow
- ☐ Measure booking completion time (end-to-end scheduling latency)
- ☐ Cache LLM router decisions in Redis